BeTrade

Project Challenges & Risk Assessment

Flutter side • Backend side • Server / Infrastructure side

Project	BeTrade — Flutter mobile client for a prediction / trading market (GHS)
Repository	github.com/Sandeeptechnobren/beTrade_App (branch: feature/sandeep)
Backend	Laravel REST API @ api.buildacademy.io (operated outside this repo)
Scope	Whole-project challenge analysis, categorised by Flutter / Backend / Server
Date	01 June 2026
Prepared for	Engineering review
Prepared by	_____
Classification	Internal / Confidential

How to read this document

Every challenge has a unique ID. **F#** = Flutter (this app), **B#** = Backend (Laravel API), **S#** = Server / Infrastructure, **X#** = Cross-cutting (spans more than one side). Backend and Server items are inferred from how the client behaves and from the project docs — the backend code is not in this repository, so those should be validated with the backend / infra owners.

Severity legend

CRITICAL	Money loss, security breach, or release blocker.
HIGH	Major reliability / compliance / UX risk.
MEDIUM	Should fix before scale; quality / maintainability.
LOW	Cleanup / polish; low risk.

System map (where the challenges live)

Executive summary

BeTrade is a functional Flutter trading client backed by an external Laravel API. The architecture is reasonable for its current scope, but several challenges must be resolved before this can operate safely as a real-money product at scale. The most urgent themes are: **financial correctness** (floating-point money, idempotency, ledger integrity), **security** (plaintext token, cleartext traffic, debug signing), **real-time pricing** (currently polled/simulated), and **release/operations maturity** (no CI/CD, no monitoring, no staging). Compliance for a Ghana-based real-money market is a leadership-level item.

Category	Critical	High	Medium	Low	Total
FLUTTER (Mobile Client) Side	3	4	8	3	18
BACKEND (Laravel API) Side	4	7	3	1	15
SERVER / Infrastructure / DevOps Side	1	6	5	0	12
Cross-Cutting & Compliance (multi-side)	2	2	0	0	4
All	**10**	**19**	**16**	**4**	**49**

Note: Backend and Server counts are challenges/risks to validate with their owners, not confirmed defects in this repo.

1. FLUTTER (Mobile Client) Side

Issues that live in this Flutter repo and are fixable by the app team.

F1 • CRITICAL

Auth bearer token stored in plaintext

What: The login bearer token is saved as a plain string in SharedPreferences, not in secure/encrypted storage.

Why it is a challenge: On a rooted/jailbroken or compromised device the token can be read and a session hijacked for a money-bearing account.

Impact: Account takeover, unauthorised trades/withdrawals.

Evidence: `lib/data/services/local_storage.dart:14-19 (setToken/getToken on SharedPreferences)`

Recommended direction: Migrate to flutter_secure_storage (Keychain / Keystore-backed).

F2 • CRITICAL

All money handled as floating-point (double)

What: Wallet balance, costs, fees, share prices and P&L are stored and computed as Dart double, with no decimal/minor-unit type and no rounding policy.

Why it is a challenge: Binary floating point cannot represent decimal currency exactly; repeated arithmetic accumulates rounding error.

Impact: Penny mismatches between app and backend, wrong displayed balances/P&L, user disputes in a real-money product.

Evidence: `quote_model.dart, order_model.dart, position_model.dart (double fields, '?? 0.0'); wallet_provider.dart:18,71`

Recommended direction: Use integer minor units (pesewas) or a Decimal type end-to-end; display-only formatting with toStringAsFixed.

F3 • CRITICAL

Idempotency key regenerated on every Confirm tap

What: A fresh random UUID v4 is generated inline each time the user taps Confirm/Buy, instead of one stable key per logical order.

Why it is a challenge: If a buy times out and the user taps again, the new key looks like a brand-new order to the backend.

Impact: Duplicate orders / double charge on retry — the exact scenario idempotency keys exist to prevent.

Evidence: `trade_buy_service.dart:63-75 (generateIdempotencyKey); buy_bottom_sheet.dart:87 (generated inside _placeBuy per tap); same pattern in wallet_provider.dart, trade_sell_service.dart`

Recommended direction: Generate one key when the order intent is created, hold it in state, and reuse it for every retry of that same order.

F4 • HIGH

No real-time prices; chart is simulated

What: Market prices update only on pull-to-refresh; the 'live' detail chart is a local Timer random-walk, not backend data.

Why it is a challenge: A prediction market is price-sensitive; users see stale or fake-moving prices.

Impact: Users place trades against outdated quotes; trust issue when filled price differs from shown price.

Evidence: `info_chart_screen.dart:59 (Timer.periodic 700ms random); no web_socket_channel/SSE anywhere; refresh via RefreshIndicator only`

Recommended direction: Add a real price feed (websocket/SSE or FCM-data push) and stop simulating; show last-updated timestamps.

F5 • HIGH

Weak client-side money validation

What: Buy/deposit/withdraw only check amount > 0 — no max, no decimal-place cap, no wallet-balance check before calling the API.

Why it is a challenge: Invalid amounts reach the backend and fail there; poor, slow feedback for the user.

Impact: Avoidable failed transactions, confusing errors, wasted round-trips, possible over-withdraw attempts.

Evidence: `buy_bottom_sheet.dart (no amount validation); newDeposit.dart:138-141,611-617; withdrawal.dart:102-105,506-510 (only >0)`

Recommended direction: Validate min/max, decimal places and available balance client-side before submit; mirror backend rules.

F6 • HIGH

Tests broken, coverage ~0%

What: The only test is the default Flutter counter scaffold, which fails because the app has no counter UI.

Why it is a challenge: No automated safety net for a money app; regressions ship silently.

Impact: Financial-logic bugs (quotes, fees, balances) can reach production undetected.

Evidence: `test/widget_test.dart (default scaffold; flutter test fails on main); no mocking/integration setup`

Recommended direction: Delete scaffold; add mocktail + integration_test; unit-test services and money math; gate CI on tests.

F7 • HIGH

Cleartext HTTP allowed + no certificate pinning

What: Android manifest enables usesCleartextTraffic=true and there is no TLS certificate pinning on Dio.

Why it is a challenge: Allows plain-HTTP fallback and makes man-in-the-middle interception easier on hostile networks.

Impact: Token/credential/trade interception or tampering on untrusted Wi-Fi.

Evidence: `android/app/src/main/AndroidManifest.xml:18 (usesCleartextTraffic="true"); no pinning in dio_client.dart`

Recommended direction: Set cleartext to false for release; add certificate pinning for the API host.

F8 • MEDIUM

Two HTTP clients coexist (dio + http)

What: Some flows use the DioClient singleton, others use the raw http package and the legacy sign-in provider.

Why it is a challenge: Divergent timeout, error-handling, and auth-header behaviour across the app.

Impact: Inconsistent failures and harder maintenance/debugging.

Evidence: `dio primary via DioClient; http used in some services + signin_provider (per audit §2/§11)`

Recommended direction: Standardise on Dio/DioClient; remove http usage.

F9 • MEDIUM

No connectivity check, retry, or offline cache

What: Failed network calls mostly return [] or null; there is no connectivity detection, no retry, no cached data.

Why it is a challenge: Offline or flaky-network users see empty screens with no explanation.

Impact: Looks like data loss / broken app; no graceful degradation.

Evidence: `services catch -> return []/null (e.g. trade_service.dart); no connectivity_plus; only token/theme cached`

Recommended direction: Add connectivity awareness, bounded retry with backoff, and a light read cache for key screens.

F10 • MEDIUM

No crash reporting or analytics on the client

What: No Crashlytics/Sentry and no product analytics are integrated.

Why it is a challenge: Production crashes and drop-offs are invisible to the team.

Impact: Field issues (especially around money flows) cannot be detected or diagnosed.

Evidence: `pubspec.yaml has no Crashlytics/Sentry/analytics packages (confirmed in audit §10)`

Recommended direction: Add crash reporting (Crashlytics or Sentry) and minimal funnel analytics.

F11 • MEDIUM

Abrupt session expiry, no token refresh

What: A 5-minute timer validates the token; on expiry the user is bounced to login. There is no refresh-token flow.

Why it is a challenge: Tokens simply die; mid-trade expiry interrupts the user.

Impact: A session can expire during an active trade/deposit; poor UX and possible half-finished actions.

Evidence: `main_screen.dart:58 (Timer.periodic 300s -> verifyToken -> Session Expired dialog); no refresh path`

Recommended direction: Add refresh tokens / sliding expiry; preserve in-progress action and re-auth gracefully.

F12 • MEDIUM

Provider sprawl, no central state contract

What: ~14 global ChangeNotifiers with public mutable fields and ad-hoc isLoading/error per provider; no single source of truth.

Why it is a challenge: State can drift between providers (e.g. wallet balance shown in multiple places).

Impact: Inconsistent UI, harder to reason about as features grow (rankings, achievements added recently).

Evidence: `MultiProvider in main.dart; per-provider isLoading/error pattern (audit §2/§3)`

Recommended direction: Define a consistent state/result pattern; consider a thin domain layer for shared values like balance.

F13 • MEDIUM

Pagination only in Explore

What: Explore supports paged loading; the home market list fetches a single page with no pagination UI.

Why it is a challenge: As the number of markets grows, a single fetch becomes heavy or truncated.

Impact: Slow/incomplete lists, higher payloads, poor scroll experience at scale.

Evidence: `explorer_provider.dart:32-107 (page/hasMore/loadMore); home list = single fetch (home_service/trade_provider)`

Recommended direction: Apply the Explore pagination pattern to all long lists.

F14 • MEDIUM

FCM payload-driven navigation not hardened

What: Push messages can drive navigation/local notifications; payload origin/shape is trusted.

Why it is a challenge: A malformed or spoofed payload could route the user or render unexpected content.

Impact: Phishing-style navigation or crashes from unexpected payloads.

Evidence: `notification_services.dart + main.dart background handler; navigatorKey used for FCM nav`

Recommended direction: Validate payload schema and source before acting; whitelist navigation targets.

F15 • MEDIUM

No build flavors (dev / staging / prod)

What: A single BASE_URL and one build configuration; no flavor split.

Why it is a challenge: Cannot safely point the app at a staging backend or ship environment-specific builds.

Impact: Testing tends to happen against production; risky for a money app.

Evidence: `.env single key BASE_URL; no flavor setup (audit §9 / ARCHITECTURE §10)`

Recommended direction: Add dev/staging/prod flavors with per-flavor config and app IDs.

F16 • LOW

Unstructured logging via ~202 print statements

What: Error handling relies on print/debugPrint throughout services.

Why it is a challenge: No log levels/redaction; sensitive data may be printed; nothing reaches a backend sink.

Impact: Noisy logs, possible PII/token leakage in device logs, no production visibility.

Evidence: `~202 print/debugPrint across services (audit §6/§11)`

Recommended direction: Introduce a logger with levels + redaction; forward errors to crash reporting.

F17 • LOW

Heavy dead code and filename inconsistencies

What: ~900+ lines of commented-out code across 6 files; a double-dot filename and several naming breaks.

Why it is a challenge: Slows comprehension and onboarding; the typo filename is imported by 7 files.

Impact: Maintainability drag; risky coordinated rename.

Evidence: `country_provider.dart (~457 commented), custom_camera.dart (~52%); api_endpoint..dart (double dot, 7 importers)`

Recommended direction: Remove dead code after git capture; rename files in one coordinated change.

F18 • LOW

Rankings UI shipped without backend data

What: The Rankings/leaderboard screen is wired in the UI but no leaderboard endpoint exists; it shows a placeholder.

Why it is a challenge: Feature appears in the app but cannot function.

Impact: Empty/placeholder feature visible to users; expectation gap.

Evidence: `rankings screen present (recent commits 90b0714/36357eb); no /leaderboard endpoint`

Recommended direction: Hide behind a flag until the backend endpoint is delivered, or stub clearly as 'coming soon'.

2. BACKEND (Laravel API) Side

Inferred from client behaviour; the API lives at api.buildacademy.io (outside this repo). To validate with the backend team.

B1 • CRITICAL

Server-side idempotency must be enforced and robust

What: Because the client can send a new key per retry (see F3), the backend must independently deduplicate orders/transfers.

Why it is a challenge: Without server-side dedupe, retries create duplicate fills and double debits.

Impact: Direct financial loss and ledger corruption.

`Evidence: Inferred from client buy/deposit/withdraw sending idempotency_key (trade_buy_service.dart:24)`

Recommended direction: Persist idempotency keys server-side with a uniqueness window; return the original result on replay.

B2 • CRITICAL

LMSR pricing correctness under concurrency

What: Quote and fill prices come from an LMSR engine; simultaneous buys on the same market/outcome must serialise.

Why it is a challenge: Races can produce inconsistent prices, over-fills, or negative liquidity.

Impact: Mispriced trades, unfair fills, financial exposure for the operator.

`Evidence: POST /trade/{uuid}/quote and /buy return LMSR fields (QuoteModel/OrderModel) — engine is backend-owned`

Recommended direction: Serialise per-market price updates (locking/transactions); validate quote-vs-fill drift server-side.

B3 • CRITICAL

Wallet ledger integrity and atomicity

What: Balance debits/credits for buys, sells, deposits, withdrawals and settlement must be atomic and consistent.

Why it is a challenge: Partial updates or non-atomic writes corrupt balances; no negative balances allowed.

Impact: Lost/extra funds, irreconcilable accounts, disputes.

`Evidence: Wallet endpoints (/wallet, /deposit, /withdraw) + buy/sell affect balance_ghs (wallet_provider.dart:71)`

Recommended direction: Double-entry ledger, DB transactions, balance constraints, and a reconciliation/audit job.

B4 • CRITICAL

Money precision on the backend / database

What: The backend and DB must store money as integer minor units or DECIMAL — never float — and agree with the client.

Why it is a challenge: If the server stores floats (matching the client double), rounding errors enter the source of truth.

Impact: Money that does not add up; cumulative drift across millions of operations.

`Evidence: Client sends/receives double GHS values (see F2); backend representation unknown`

Recommended direction: Confirm DECIMAL/minor-unit storage and a single rounding policy shared with the client.

B5 • HIGH

Market settlement / resolution engine

What: When a market resolves, winners must be computed and paid out correctly and atomically.

Why it is a challenge: Settlement is the highest-stakes money event in a prediction market.

Impact: Wrong or double payouts, missed payouts, disputes and reputational damage.

`Evidence: PositionModel has is_winner/realizedPnl; resolution/payout logic is backend-owned`

Recommended direction: Deterministic, audited, replay-safe settlement with idempotent payouts and an audit trail.

B6 • HIGH

Token lifecycle: refresh, expiry, revocation

What: Backend issues bearer tokens with no client refresh; TTL and revocation policy are backend-defined.

Why it is a challenge: Drives the abrupt logout UX (F11) and governs session security.

Impact: Either sessions die mid-action, or long-lived tokens raise hijack risk.

Evidence: `/verify-token, /logout; no refresh endpoint consumed by client`

Recommended direction: Add refresh tokens with rotation, server-side revocation on logout, sensible TTLs.

B7 • HIGH

Rate limiting and abuse protection

What: OTP, quote, buy and withdraw endpoints need throttling and abuse detection.

Why it is a challenge: OTP without limits = SMS-cost abuse / bombing; quote scraping; transaction spam.

Impact: Runaway SMS spend, scraping of pricing, fraud/automation attacks.

Evidence: `OTP endpoints (/register,/login,/verify-otp/*); high-frequency /quote from typing debounce (trade_page.dart:167)`

Recommended direction: Per-IP/per-user rate limits, OTP cooldowns/attempt caps, anomaly detection.

B8 • HIGH

KYC verification pipeline and gating

What: The client uploads KYC docs; the backend must verify, track status, and gate trading/withdrawal accordingly.

Why it is a challenge: Required for AML/compliance in a real-money product.

Impact: Non-compliant onboarding; unverified users transacting.

Evidence: `POST /kyc/submit (multipart) from verify_account.dart; KYC_REQUIRED code referenced in client`

Recommended direction: Define KYC states, a verification workflow (vendor or manual), and enforce gating server-side.

B9 • HIGH

No API versioning

What: Endpoints are unversioned; installed apps cannot be force-updated instantly.

Why it is a challenge: A breaking change to a response shape will break older app versions in the field.

Impact: Field outages for users who have not updated; risky deploys.

Evidence: `Flat paths under /projects/betrade/public/api (no /v1)`

Recommended direction: Introduce versioned routes and a deprecation policy; pair with a client min-version gate (S13).

B10 • HIGH

No server-side real-time price publishing

What: There is no stream/websocket for price updates; the client can only poll.

Why it is a challenge: Real-time pricing (F4) cannot exist without a backend publisher.

Impact: Stale prices for all clients; unfair fills.

Evidence: `No realtime channel besides FCM (ARCHITECTURE §11)`

Recommended direction: Stand up a price-publish channel (websocket/SSE) or push price deltas via FCM-data.

B13 • HIGH

Deposit / withdraw payment integration (GHS)

What: Wallet funding is fully backend-driven; for GHS this almost certainly means mobile money / bank rails with webhooks.

Why it is a challenge: Money movement needs reliable provider integration, callbacks, and reconciliation.

Impact: Stuck/duplicated deposits, failed withdrawals, reconciliation gaps.

Evidence: `/wallet/deposit, /wallet/withdraw exist; no client payment SDK (audit §10) -> all server-side`

Recommended direction: Confirm provider (e.g. mobile money), idempotent webhooks, retries, and reconciliation reports.

B11 • MEDIUM

Error-code contract must be stable and exhaustive

What: Client maps typed codes (INSUFFICIENT_FUNDS, KYC_REQUIRED, MARKET_CLOSED, BELOW_MIN_COST...) to messages.

Why it is a challenge: Unknown or renamed codes fall through to generic errors.

Impact: Confusing errors; missed business rules surfaced as 'something went wrong'.

Evidence: `buy_bottom_sheet.dart:104-123 (_formatError maps codes)`

Recommended direction: Publish and version the full error-code catalogue; keep it stable.

B12 • MEDIUM

Response-shape stability vs manual parsing

What: The client hand-writes fromJson with defensive defaults; any silent field rename degrades data.

Why it is a challenge: No schema contract; mismatches fail quietly to 0.0/empty.

Impact: Wrong values shown with no error (e.g. balance becomes 0.0).

Evidence: `Manual fromJson with '?? 0.0' across models (audit §4)`

Recommended direction: Document and version response schemas; consider a contract test between app and API.

B14 • MEDIUM

Authorization audit (server-enforced)

What: All gating is server-side with a single user role; per-route authorization should be reviewed.

Why it is a challenge: Client cannot be trusted; every money route must verify ownership and KYC.

Impact: Potential for accessing/acting on another user's resources if a route is under-protected.

Evidence: `Authorization enforced server-side (ARCHITECTURE §6)`

Recommended direction: Audit every authenticated route for ownership + KYC checks; add tests.

B15 • LOW

Defined-but-unused endpoints

What: Several endpoints are defined client-side but never called (/languages, /notificationPreferences, /chart).

Why it is a challenge: Dead surface area creates confusion about what is supported.

Impact: Maintenance noise; unclear contract.

Evidence: `ApiEndpoints.languages / notificationPreferences; /trade/{uuid}/chart fetcher never called (audit §11)`

Recommended direction: Confirm intent; remove or wire up.

3. SERVER / Infrastructure / DevOps Side

Build, release, hosting, security and operations.

S1 • CRITICAL

No release signing setup (debug keystore / no iOS team)

What: Android release builds are signed with the debug keystore; iOS has no development team or provisioning profile.

Why it is a challenge: Debug-signed builds are not publishable and are insecure; iOS cannot distribute at all.

Impact: Cannot ship to Play Store / App Store; security risk if a debug build leaks.

Evidence: `android/app/build.gradle.kts:33-34 (signingConfigs.getByName("debug")); no key.properties/.jks; iOS team absent`

Recommended direction: Create a release keystore + key.properties (kept out of git); configure iOS signing per CI/dev.

S2 • HIGH

No CI/CD on main

What: There is no GitHub Actions or Codemagic pipeline on main; Codemagic exists only on unmerged branches.

Why it is a challenge: No automated build/test/lint gate; releases are manual and non-reproducible.

Impact: Inconsistent builds, no test enforcement, slow and error-prone releases.

Evidence: `.github/workflows none on main; codemagic.yaml only on feature branches (audit §8)`

Recommended direction: Add CI on main: analyze + test + build; later automate signed TestFlight/Play uploads.

S3 • HIGH

No monitoring, alerting, or crash reporting

What: Neither client crash reporting nor backend APM/alerting is in place (from what is visible).

Why it is a challenge: The team is blind to production failures, especially around money flows.

Impact: Incidents discovered via user complaints, not telemetry; slow response.

Evidence: `No Crashlytics/Sentry in pubspec (audit §10); backend observability not visible`

Recommended direction: Add client crash reporting and backend metrics/log-based alerts on error rates and payment failures.

S4 • HIGH

Secrets / config management gaps

What: Firebase config files (google-services.json, GoogleService-Info.plist) are committed; release used a debug keystore.

Why it is a challenge: Committed config and keys widen the exposure surface; no secret vault.

Impact: Leaked keys/config; harder rotation; signing-material risk.

Evidence: `.gitignore excludes .env (good) but NOT the Firebase config files; debug keystore in repo flow`

Recommended direction: Gitignore Firebase config + keystores, distribute via CI secrets, verify Firebase Security Rules.

S5 • HIGH

No environment split (dev / staging / prod)

What: A single backend URL and no staging environment.

Why it is a challenge: There is nowhere safe to test changes before they hit real users/money.

Impact: Testing on production; high blast radius for mistakes.

Evidence: `Single BASE_URL (.env); no staging documented (ARCHITECTURE §10)`

Recommended direction: Provision a staging backend + matching app flavor (pairs with F15).

S6 • HIGH

Backend host ownership / SLA unknown

What: The backend and any server are operated outside this repo; ownership and access docs are full of [FILL IN].

Why it is a challenge: No clear owner, SLA, escalation path, or provisioned server for ops.

Impact: Bus-factor and operational risk; nobody clearly accountable for uptime/incidents.

Evidence: `ACCESS.md / SSH_CONFIG.md [FILL IN]; 'no server provisioned yet'`

Recommended direction: Document owner, SLA, on-call and access; provision and record the server target.

S7 • HIGH

Network security at the infrastructure layer

What: The platform must enforce HTTPS/HSTS with valid certs; SSH restricted to known IPs/VPN.

Why it is a challenge: Cleartext is allowed client-side (F7); infra must not permit downgrade.

Impact: MITM/interception if TLS or firewalling is weak.

Evidence: `usesCleartextTraffic=true on client; firewall/VPN only documented, not provisioned (SSH_CONFIG.md)`

Recommended direction: Enforce TLS/HSTS, disable cleartext, lock SSH to VPN/known IPs, rotate keys.

S8 • MEDIUM

Backend scaling and availability unknown

What: Read load (markets/quotes/positions) and write contention (buys/settlement) scaling behaviour is unverified.

Why it is a challenge: Trading bursts (popular markets, deadlines) create spikes.

Impact: Latency/outages during peaks; failed trades at the worst moment.

Evidence: `Single backend host; no load-testing evidence in repo`

Recommended direction: Load test; add caching/read replicas and autoscaling; capacity plan around market deadlines.

S9 • MEDIUM

Database operations (backups / migrations / DR)

What: The money-critical DB (wallet ledger, orders) needs backups, point-in-time recovery, and safe migrations.

Why it is a challenge: Data loss in a financial ledger is catastrophic and may be unrecoverable.

Impact: Permanent loss of balances/history; inability to reconcile.

Evidence: `Backend DB is external/not visible (ARCHITECTURE §4/§12)`

Recommended direction: Confirm automated backups + PITR, tested restores, and reviewed migrations.

S10 • MEDIUM

No force-upgrade / kill-switch for clients

What: There is no server-driven minimum-version gate to retire buggy app versions.

Why it is a challenge: A bad shipped build (money bug) cannot be quickly disabled.

Impact: A defective client keeps transacting until users update voluntarily.

Evidence: `No min-version/remote-config check at startup`

Recommended direction: Add a startup version/health check that can soft-block or force-update.

S11 • MEDIUM

Two Firebase projects (config drift risk)

What: Mobile uses betrade-new while web/desktop config references betrade-4efd1.

Why it is a challenge: Wrong-project config can send pushes to the wrong place or fail silently.

Impact: Mis-delivered/lost notifications; confusing setup.

Evidence: `firebase_options.dart references both projects (ARCHITECTURE §2/§8)`

Recommended direction: Consolidate to one project (or clearly separate prod/test) and document ownership.

S12 • MEDIUM

No centralized logs / financial audit trail

What: No centralized logging or immutable audit trail for money actions is evident.

Why it is a challenge: Disputes and compliance require a tamper-evident record of every financial event.

Impact: Cannot prove what happened during a dispute or audit.

Evidence: `Client uses print only; backend audit logging not visible`

Recommended direction: Centralize logs; keep an append-only audit log for orders/wallet/settlement with retention.

4. Cross-Cutting & Compliance (multi-side)

Challenges that span Flutter + Backend + Server and need coordinated ownership.

X1 • CRITICAL

Regulatory & legal exposure (real-money, GHS market)

What: This is a real-money prediction/trading product priced in Ghanaian Cedi; it likely falls under financial and/or gaming regulation.

Why it is a challenge: Real-money trading/prediction markets typically require licensing, AML/KYC, age-gating, responsible-gaming controls and clear T&Cs.

Impact: Legal/operational shutdown risk, fines, payment-processor refusal if non-compliant.

Evidence: `GHS currency throughout; wallet deposit/withdraw + trading; KYC flow present`

Recommended direction: Get a legal/compliance review for Ghana (Bank of Ghana / SEC / Gaming Commission as applicable); implement required controls. [Owner: leadership/legal]

X2 • CRITICAL

End-to-end financial integrity (cross-team)

What: Money correctness spans client (F2/F3/F5), backend (B1-B4) and DB (S9): precision, idempotency, ledger atomicity.

Why it is a challenge: A weakness in any layer breaks the whole money guarantee.

Impact: Financial loss, corrupted balances, loss of user trust.

Evidence: `See F2,F3,F5,B1,B2,B3,B4,S9`

Recommended direction: Treat as one workstream with a shared money spec (units, rounding, idempotency, ledger invariants).

X3 • HIGH

End-to-end real-time pricing (cross-team)

What: Live pricing needs a client consumer (F4), a backend publisher (B10) and infra to scale it (S7/S8).

Why it is a challenge: None of the three exist today; prices are polled/simulated.

Impact: Stale prices, unfair fills, weak product feel for a trading app.

Evidence: `See F4, B10, S8`

Recommended direction: Design the streaming architecture across all three layers together.

X4 • HIGH

End-to-end KYC / identity (cross-team)

What: KYC spans client capture (camera), backend verification (B8) and possibly a third-party vendor + storage.

Why it is a challenge: Identity is a compliance gate for funding and trading.

Impact: Non-compliant onboarding; PII handling/storage risk.

Evidence: `See F-capture (camera/KYC), B8`

Recommended direction: Define the full KYC journey, vendor (if any), data retention and gating end-to-end.

Suggested sequencing (high level)

Phase	Focus items
Wave 1 — Stop the bleeding (before any real-money pilot)	F1 secure token, F3 idempotency key reuse, F7 cleartext off, S1 release signing, B1 server idempotency, B3 ledger atomicity, B4 + F2 money precision spec, X1 compliance review kickoff.
Wave 2 — Trust & safety	F5 money validation, F6 tests + CI (S2), S3 crash/monitoring, B6 token refresh, B7 rate limiting, B8 KYC pipeline, S4 secrets, S5/F15 staging + flavors.
Wave 3 — Product depth & scale	F4 + B10 + X3 real-time pricing, B5 settlement hardening, B13 payment integration, F13 pagination, S8 scaling, S9 DB backups/DR, B9 API versioning, S10 force-upgrade.
Wave 4 — Polish	F8 single HTTP client, F9 offline/retry, F16 logging, F17 dead code, F18 rankings gating, B15 unused endpoints, S11 Firebase consolidation, S12 audit logs.

This sequencing is a suggestion for discussion, not a commitment. Effort sizing and exact ownership (app team vs backend team vs infra/leadership) should be agreed in the review.